

Stage 1 Tabular Machine Learning

End-to-End Data Engineering Architecture & Operational Technical Specification

EXECUTIVE SUMMARY & ROLE SCOPE:

As the **Data Engineer in Stage 1 Tabular Machine Learning**, the core responsibility is designing, building, and maintaining an enterprise-grade, reproducible data infrastructure. This encompasses raw clinical data ingestion, patient-level zero-leakage splitting, automated missing data imputation, domain-specific feature engineering (producing 66 numerical/categorical features), feature selection, and real-time transformation serving for model training and deployment.

1. Data Pipeline System Architecture

The Stage 1 data pipeline processes raw patient clinical records and converts them into normalized, encoded feature matrices consumed by XGBoost, CatBoost, and Random Forest classifiers. Below is the end-to-end flow:

- **Raw Clinical Ingestion:** Pulling patient demographics, laboratory vitals, comorbidities, and therapy dosage logs.
- **Group-Stratified Splitter:** Enforcing patient-level separation to isolate multi-visit records strictly across splits.
- **Missing Value Imputation:** Dual-strategy median/mode imputers avoiding statistical data leakage.
- **Domain Feature Factory:** Generating 66 domain-engineered features (clinical risk indices, interaction terms, dosage intensity).
- **Feature Selection & Variance Pruning:** Removing constant features via VarianceThreshold.
- **Encoding & Scaling:** One-Hot Encoding categorical descriptors and StandardScaling continuous lab variables.
- **Artifact Serialization:** Exporting reusable transformers (`.joblib`) for zero-latency API inference.

2. Data Ingestion & Zero-Leakage Partitioning

Longitudinal medical datasets frequently contain multiple visits per patient. A naive random split leads to severe patient leakage, inflating validation metrics artificially. The Data Engineer implemented a patient-stratified partitioning strategy:

Partition	Patient Count	Record Count	Percentage	Leakage Guarantee
Train Set	2,560	7,896	80.0%	0.00% Patient Overlap
Validation Set	320	1,010	10.0%	0.00% Patient Overlap
Holdout Test Set	320	950 / 750 (Unseen)	10.0%	0.00% Patient Overlap

3. Data Cleaning & Imputation Infrastructure

Clinical raw data exhibits missing values due to uncollected lab panels or varying hospital protocols. The Data Engineer built a dual-channel imputation engine using Scikit-Learn transformers fit strictly on training partitions:

1. Numerical Variables (Median Imputation):

Continuous variables such as age, BMI, blood pressure, ALT/AST liver enzymes, and tumor size are imputed using `SimpleImputer(strategy='median')`. Median is selected over mean to remain robust against extreme physiological outliers.

2. Categorical Variables (Mode Imputation):

Discrete descriptors such as tumor stage, histological grade, and treatment regimen are imputed using `SimpleImputer(strategy='most_frequent')`.

4. Oncology Feature Engineering Framework

The centerpiece of Stage 1 ML is `feature_engineering.py`, which translates raw physiological metrics into **66 clinically meaningful features**. The key engineered domains include:

Feature Domain	Engineered Features / Derivation Formula	Clinical Rationale
Age Binning	age_group: (<50, 50-65, >65)	Captures non-linear risk escalation in elderly oncology cohorts.
BMI Profiling	bmi_category: (underweight, normal, overweight, obese)	Accounts for cachexia vs obesity-related toxicity risks.
Tumor Size Bins	tumor_size_category: T1 (≤2cm), T2 (2-5cm), T3 (>5cm)	Standardized AJCC tumor T-stage categorization.
Treatment Intensity	treatment_intensity = dose / duration	Measures daily therapeutic burden and potential drug toxicity.
Clinical Composite Risk	high_clinical_risk: Binary flag (High Comorbidity & Poor ECOG)	Identifies frailty in patients with multi-organ comorbidities.
Biomarker Interactions	biomarker_interaction = biomarker_1 * biomarker_2	Models synergistic gene expression interactions.

5. Preprocessing, Encoding & Serialization

After feature creation, data undergoes rigid statistical formatting before being saved as reusable pipeline artifacts:

- **Variance Thresholding:** `VarianceThreshold(threshold=0.0)` drops zero-variance constant features.
- **One-Hot Encoding:** `OneHotEncoder(handle_unknown='ignore', sparse_output=False)` converts categorical strings to dense binary vectors.
- **Standard Scaling:** `StandardScaler()` normalizes continuous lab values to zero mean and unit variance ($Z = \frac{X - \mu}{\sigma}$).

- **Artifact Serialization:** Imputers, encoders, and scalers are saved using `joblib.dump()` to `data/stage1_ml/models/`.

6. API & Real-Time Production Integration

The Data Engineer designed the real-time inference input transformer embedded inside `OncologyPredictionPipeline` (`[prediction.py](file:///c:/Users/ADMIN/Desktop/oncology/personalized_precision_oncology/stage1_ml/prediction/prediction.py)`):

1. **Real-Time API Payload Processing:** Receives raw single-patient JSON via FastAPI endpoint `POST /predict`, runs identical imputation & feature engineering transformations in-memory under **5 ms latency**.
2. **Batch CSV Pipeline:** Processes multi-patient CSV files for batch clinical decision support in Streamlit dashboard (`integration/dashboard/app.py`).

7. Data Quality Assurance & Unit Tests

To prevent pipeline breakage in production, automated test suites verify data contracts (`[test_stage1.py](file:///c:/Users/ADMIN/Desktop/oncology/personalized_precision_oncology/stage1_ml/tests/test_stage1.py)`):

- **Schema Contract Assertion:** Validates all 66 target feature columns exist in processed DataFrames.
- **Zero Null Assertion:** Asserts zero NaN or infinite values after pipeline execution.
- **Type Checking:** Verifies floating-point precision alignment for PyTorch/XGBoost inputs.
- **Boundary Validation:** Ensures probabilities remain bounded in $[0.0, 1.0]$.